

Sliding Windows and Sparse Attention

Skipping pairs on purpose, and what it costs · video 2:48

This lesson answers one question: what happens if attention skips most pairs? With a **sliding window**, each token sees only the last W tokens, so the work grows like $T \times W$ instead of T^2 . Stacked layers let information hop further back, in theory. In practice, a tiny model with a window did *better* on Shakespeare, but completely failed a task that needs exact recall from 128 tokens back. That is why models mix windowed and full attention layers.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The window mask	▶ 0:08-0:40	2	☐
2. Reaching further by stacking	▶ 0:40-1:04	2	☐
3. Two experiments, two answers	▶ 1:04-1:50	3	☐
4. Mixing, sparse patterns, and the cache	▶ 1:50-2:39	2	☐

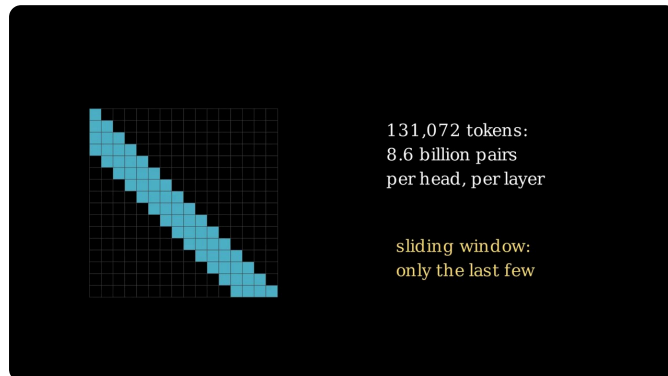
Before you start: Deep Dive episodes 7 (causal mask) and 10 (FlashAttention). Code: `code/d11_sliding_window` (PyTorch only; trains six tiny models, about 20 minutes on a CPU).

CONCEPT 1 OF 4

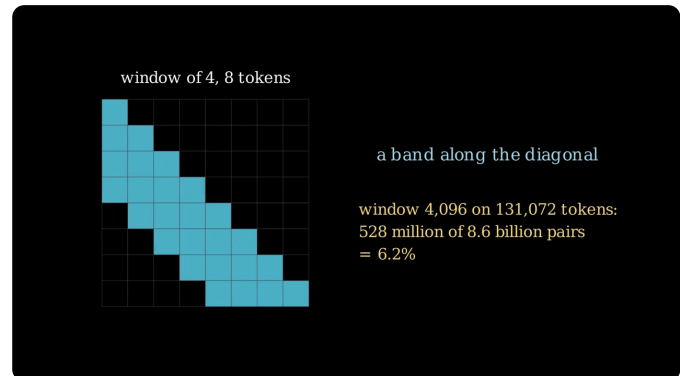
The window mask

▶ Video 0:08–0:40

Full attention compares every token with every earlier one, so the work grows with the square of the length: at 131,072 tokens that is **8.6 billion** pairs, per head, per layer. A **sliding window** lets each token look only at the last W tokens (itself included).



▶ 0:24 Full causal attention is a triangle of pairs; a sliding window keeps only the last few.



▶ 0:39 A window of 4 on 8 tokens: a band along the diagonal. Window 4,096 at 131,072 tokens: 6.2% of the pairs.

The mask becomes a band along the diagonal. With $W = 4,096$ on 131,072 tokens, 528 million pairs remain: **6.2%**.

KEY IDEA

A window turns the quadratic cost T^2 into roughly $T \times W$: linear in the length.

Check 1: The window mask

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

How many query-key pairs does a window of 128 allow on 1,024 tokens (full causal: 524,800)?

1.2 TRUE OR FALSE

“With a window of 4, token 7 can attend to token 3.”

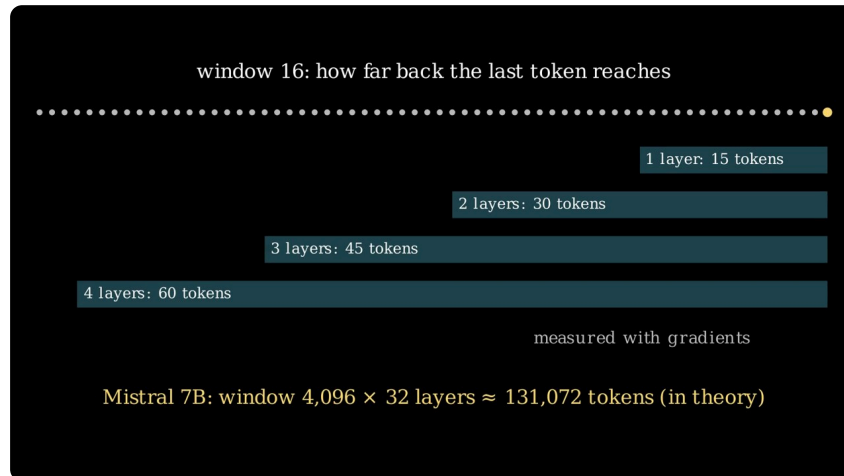
True ☐ False ☐

CONCEPT 2 OF 4

Reaching further by stacking

▶ Video 0:40–1:04

Information can hop. With a window of 16, one layer lets the last token reach 15 tokens back; the next layer reaches through those, and so on: **15, 30, 45, 60** after 1–4 layers, measured by checking which inputs receive a gradient from the last output. Mistral 7B uses a window of 4,096 with 32 layers: in theory, a reach of about 131,072 tokens.



▶ 1:02 With a window of 16, each layer adds 15 tokens of reach: 15, 30, 45, 60, measured with gradients.

KEY IDEA

Reach grows with layers × (W – 1), like a receptive field in a convolutional network, but reach is only a possibility, not a guarantee.

Check 2: Reaching further by stacking

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

A model has 12 layers with a window of 512. What is its theoretical reach?

.....

2.2 SHORT ANSWER

Why measure reach with gradients rather than by looking at attention weights?

.....

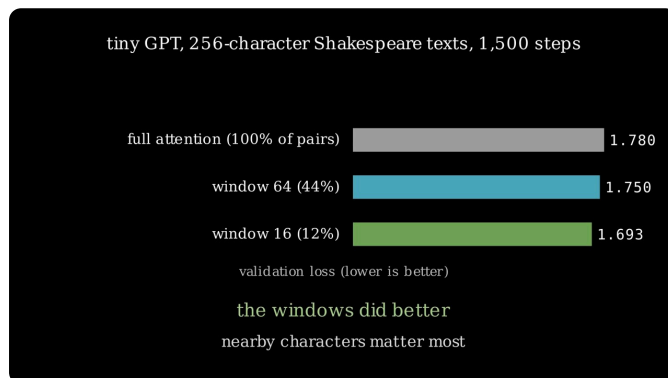
CONCEPT 3 OF 4

Two experiments, two answers

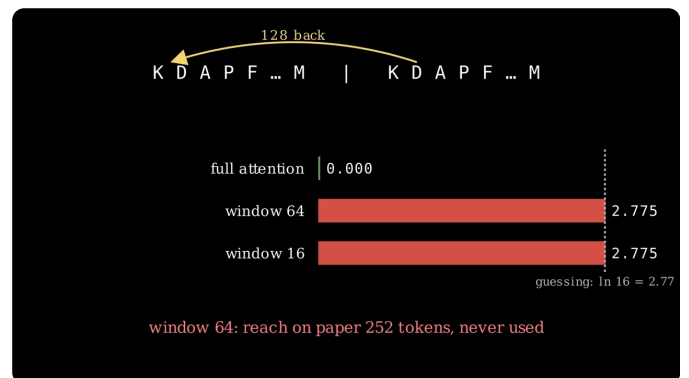
▶ Video 1:04-1:50

The same tiny GPT is trained for 1,500 steps on 256-character Shakespeare texts. Validation loss: full attention **1.780**, window 64 **1.750**, window 16 **1.693**, with 44% and 12% of the pairs. The windows did better: for spelling out Shakespeare character by character, nearby characters matter most, and the window is a helpful shortcut for a small model trained briefly.

Now a task that needs long range: 128 random symbols, then the same 128 again. Full attention learns to copy perfectly: loss **0.000** on the second half. Both windows are stuck at **2.775**, pure guessing ($\ln 16 = 2.77$). Even window 64, whose reach on paper is 252 tokens, never learned to relay the symbols through its layers.



▶ 1:27 Shakespeare, 256 characters: full 1.780, window 64 1.750, window 16 1.693. The windows did better.



▶ 1:49 Copy 128 symbols: full attention 0.000; both windows 2.775, pure guessing ($\ln 16 = 2.77$).

KEY IDEA

Windows are cheap and fine for local patterns, but lose exact recall far back. One benchmark can hide what another reveals.

Check 3: Two experiments, two answers

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

Which task would most likely suffer from a window of 16?

- A. Predicting the next letter of a word
- B. Answering a question about the first page of a long report
- C. Closing a bracket opened 3 tokens earlier
- D. Choosing between “a” and “an”

3.2 CALCULATION

In the copy task, what loss does pure guessing among 16 symbols give?

3.3 TRUE OR FALSE

“If a window's theoretical reach covers a distance, the model will learn to use it.”

True ☐ False ☐

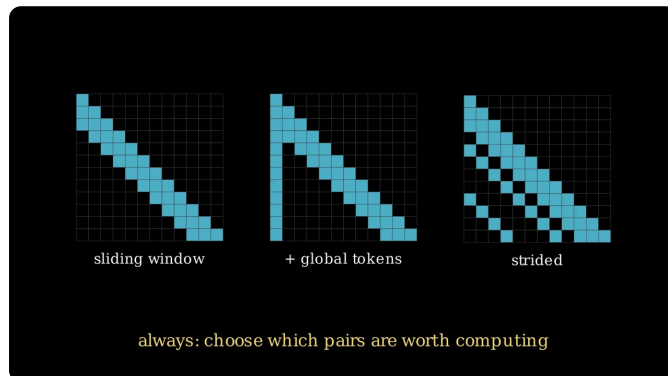
CONCEPT 4 OF 4

Mixing, sparse patterns, and the cache

▶ Video 1:50-2:39

That is why models **mix**: Gemma 2, for example, alternates sliding-window layers with full-attention layers. Sliding windows are one kind of **sparse attention**; other patterns add a few **global tokens** that see, and are seen by, everything, or skip with a **stride**. The idea is always to choose which pairs are worth computing.

At inference there is a bonus: with a window, the KV cache only needs the last W tokens, a fixed-size **rolling buffer** however long the text grows.



▶ 2:16 Other sparse patterns: global tokens, strides. Always: choose which pairs to compute.



▶ 2:38 The window mask in one line: not in the future, and less than W back.

KEY IDEA

Windowed layers handle the local work cheaply; a few full layers keep long-range recall.

Check 4: Mixing, sparse patterns, and the cache

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

With a window of 4,096, how many tokens' keys and values must a layer keep in its cache while generating token 100,000?

4.2 HANDS-ON CODE

Try it yourself. Print `window_mask(10, 3).int()`. How many ones are in it?

```
print(window_mask(10, 3).int(), window_mask(10, 3).sum())
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The window mask · rewatch ▶ 0:08-0:40

1.1 122,944 (about 23%). Token i sees $\min(i + 1, 128)$ keys; summing over 1,024 tokens gives 122,944.

1.2 False. It sees tokens 4–7 only: $7 - 3 = 4$ is not less than $W = 4$ (look at row 7 of the printed mask).

2. Reaching further by stacking · rewatch ▶ 0:40-1:04

2.1 6,132 tokens. $12 \times (512 - 1) = 6,132$, by the same rule as $4 \times 15 = 60$.

2.2 A gradient is nonzero exactly when an input can influence the output through any path, across all layers; attention weights show only one layer's direct links. The code backpropagates from the last output and counts which inputs receive a gradient.

3. Two experiments, two answers · rewatch ▶ 1:04-1:50

3.1 B. It needs exact information from far back; the others are local.

3.2 About 2.77. $\ln 16 \approx 2.773$; both windowed models scored 2.775.

3.3 False. Window 64 with 4 layers can reach 252 tokens in theory, but scored 2.775 on the copy task, no better than guessing.

4. Mixing, sparse patterns, and the cache · rewatch ▶ 1:50-2:39

4.1 4,096. Only the last W tokens can be attended to; older entries can be dropped.

4.2 27. Rows 0 and 1 have 1 and 2 ones; the other 8 rows have 3: $1 + 2 + 24 = 27$.